

Lập trình API -
Thư viện liên kết động

Nguyễn Thu Hường

Khoa CNTT – Đại học GTVT

Nội dung của chương

- Các khái niệm về DLL
- Tạo một DLL
- Sử dụng một DLL

Khái niệm

- Các thư viện liên kết động (dynamic-link libraries hay viết tắt là DLLs) là một trong các phần tử có cấu trúc quan trọng nhất của Microsoft Windows
- Các thư viện liên kết động nhìn chung không phải là các tệp thực thi, và nhìn chung chúng không nhận các thông điệp
- Chúng là các tệp tin tách biệt có chứa các hàm mà có thể được gọi bởi các chương trình và các DLLs khác để thực hiện các công việc nào đó.
- Một thư viện liên kết động chỉ hoạt động khi một module khác gọi một trong các hàm trong thư viện này.

Khái niệm

- Các thư viện sau đây đều là các thư viện liên kết động, tất cả các chương trình cho Windows đều có thể sử dụng chúng:
  - KERNEL32.DLL, USER32.DLL, và GDI32.DLL
  - Các tệp driver khác nhau như:
    - KEYBOARD.DRV, SYSTEM.DRV, và MOUSE.DRV
    - Các driver cho màn hình và máy in
- Mặc dù một module thư viện liên kết động có thể có bất cứ tên mở rộng nào (chẳng hạn như .EXE hoặc .FON), song tên mở rộng chuẩn là .DLL.
- Chỉ có các thư viện liên kết động với tên mở rộng .DLL mới được tải tự động bởi Windows.

Ưu điểm

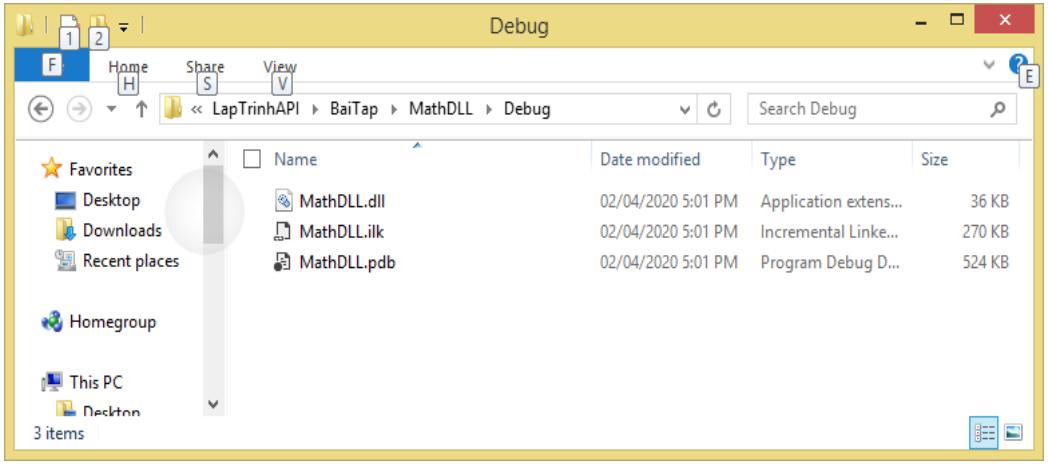
- Giảm không gian sử dụng của bộ nhớ
- Giảm swapping (thay trang bộ nhớ)
- Có thể đóng gói và đưa vào chương trình khác
- Tạo ra khả năng tương tác với các ngôn ngữ lập trình
- Dễ dàng đưa ra sự hỗ trợ sau khi đã chuyển giao ứng dụng cho khách hang

Nhược điểm

- DLL Hell:
  - Là khó khăn lớn nhất thỉnh thoảng gặp khi dùng DLL. Ta có thể gặp những thông báo lỗi dạng như: `The ordinal abc could not be located in the dynamic-link library xyz.dll`
  - Hoặc khi cài đặt một ứng dụng mới, một số chương trình khác đang bình thường bỗng nhiên bị trục trặc hoặc thậm chí không thể nạp lên để chạy được.
  - Lý do: chương trình cài đặt không kiểm tra phiên bản của các DLL trước khi sao lưu nó vào trong thư mục hệ thống. Khi một DLL mới thay thế một DLL cũ có sẵn, và nếu DLL mới này có một số thay đổi lớn làm cho nó không thể tương thích ngược lại với các chương trình sử dụng phiên bản cũ, nó sẽ làm rối loạn chương trình đó.

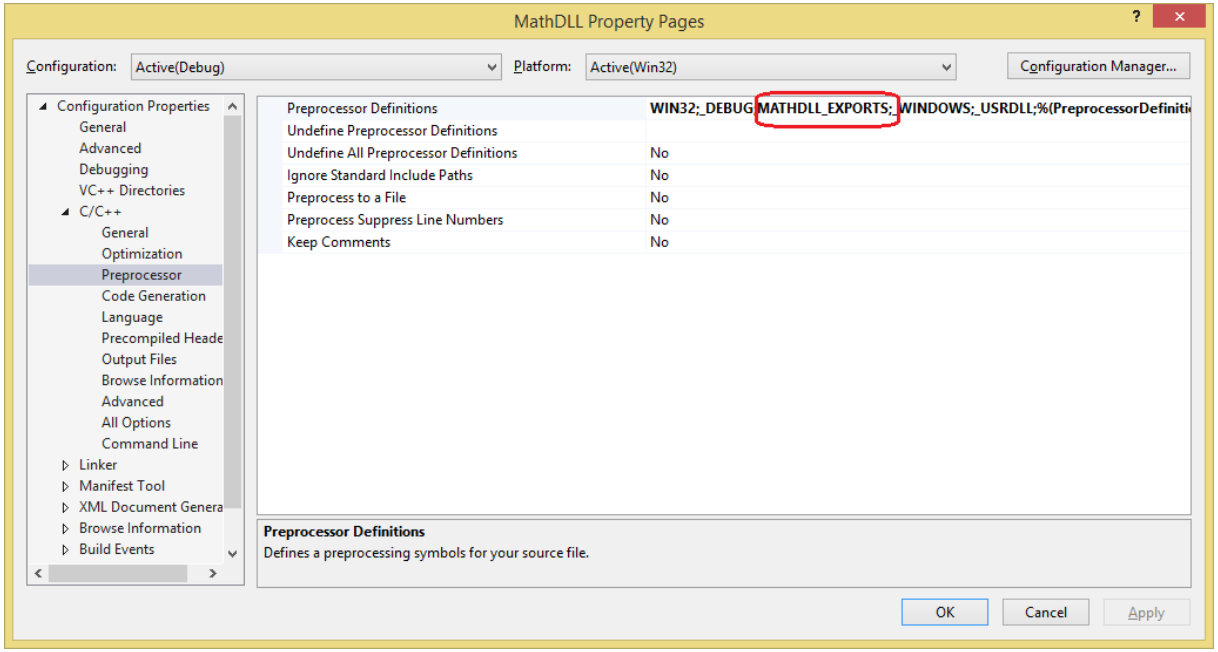
Tạo một DLL

- Tạo một thư viện DLL chứa các hàm toán học đơn giản thực hiện 4 phép tính: cộng, trừ, nhân, chia hai số nguyên bằng C, C++.
- Bước 1: Tạo ứng dụng kiểu Dynamic-Link Library (DLL). Đặt tên ứng dụng là **MathDLL**.
  - Sau khi build dự án sẽ tạo ra một file DLL trong thư mục Debug của ứng dụng.



Tạo một DLL

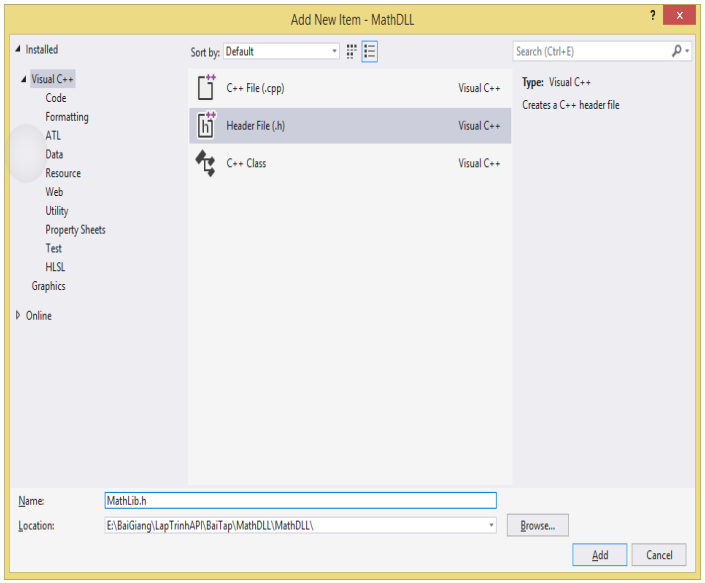
- Bước 2: Click chuột phải vào project hiện tại, chọn **Properties**. Các bạn vào **Configuration Properties -> C/C++ -> Preprocessor** và xem trong phần **Preprocessor Definations**:



Nếu là **MATHDLL_EXPORTS** trong danh sách Preprocessor Defination thì không cần chỉnh sửa gì thêm

Tạo một DLL

- Bước 2: Tạo phần header (file .h) cho thư viện. Click chuột phải vào mục Header Files của Project, chọn Add/New Item… sau đó xuất hiện hộp thoại Add new Item…. **Đặt tên file header là MathLib.h**




```
#pragma once
#ifdef  MATHDLL_EXPORTS
#define MATHDLL_API _declspec(dllexport)
#else
#define MATHDLL_API _declspec(dllimport)
#endif //  MATHDLL_EXPORTS
extern "C" MATHDLL_API int Add(int a, int b);
extern "C" MATHDLL_API int Sub(int a, int b);
extern "C" MATHDLL_API int Mull(int a, int b);
extern "C" MATHDLL_API int Div(int a, int b);
```

Tạo một DLL

- Bước 3: Xây dựng phần định nghĩa cho các chức năng trong thư viện. (Định nghĩa trong file .cpp)

```
#include "pch.h"
#include "MathLib.h"
int Add(int a,int b)
{
return a + b;
}
int Sub(int a, int b)
{
return a - b;
}
int Mul(int a, int b)
{
return a * b;
}
int Div(int a, int b)
{
return a*b;
}
```

- Cuối cùng ta chỉ cần build lại chương trình.

Sử dụng DLL

- Bây giờ ta sẽ tạo dự án để sử dụng thư viện MathDLL trên. Để sử dụng thư viện dll ta quan tâm đến các file .dll, .h và file .lib
- **Bước 1:** Tạo project win32 Application mới có tên UsingMathDll

Sử dụng DLL

- **Bước 2:** Ta coppy các file cần thiết của thư viện MathDLL vào ứng dụng.
  - Coppy file MathLib.h vào thư mục Header Files của Project
  - Coppy file MathDLL.lib vào thư mục chung của ứng dụng
  - Copy file MathDLL.dll vào thư mục chứa file exe của ứng dụng.

Sử dụng DLL

- **Bước 3:** Trong file mã nguồn UsingMathDll.cpp ta thêm khai báo **include** đến MathLib.h. Sau bước này ta có thể gọi bất cứ hàm API nào trong thư viện MathDLL đã tạo ở trên.

Sử dụng DLL

- **Bước 4:** Gọi hàm trong thư viện DLL. Ví dụ trong thông điệp WM_PAINT của ứng dụng ta gọi đến hàm Add để cộng 2 số thực trong thư viện trên như sau:

```
case WM_PAINT:
    {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hWnd, &ps);
        // TODO: Add any drawing code that uses hdc here...
        wsprintfW(ketQua,L"%d + %d=%d", 3, 4,Add(3,4));
        TextOut(hdc, 50, 50, ketQua,8);
        EndPaint(hWnd, &ps);
    }
    break;
```

  - **Trong hàm wsprintfW:** biến **ketQua** là biến mảng kiểu TCHAR để lưu trữ dòng cần in ra